

Virtue7: A Lightweight Governed Runtime for AI Work

Virtue7
August 2026

Abstract. Large language models can write, reason, and use tools, but those abilities are often exposed through one undifferentiated conversation. A request to discuss an idea, create a document, or execute a bounded task may enter the same growing context and accumulate the same instructions. Virtue7 separates language quality from the purpose and lifespan of work. The Writing Kit provides one governed language substrate and three operating modes: Chat, Content, and Automation. Chat is conversational. Content produces a bounded artifact. Automation performs a bounded task with a terminal condition. The runtime uses minimum-sufficient loading: three compact boot files, then only the route, profile, state, or tool contract required for the current step. Evidence class, form, cadence, privacy, tool truth, artifact state, and human approval remain stable across those modes.

1. Introduction

The practical failure mode of many AI writing systems is not simply weak prose. It is context without architecture. Instructions for brainstorming, drafting, research, personal voice, tool use, revision, and publication accumulate in one place even though they govern different kinds of work. The model then spends attention on machinery that the current task does not need.

Virtue7 treats context as an execution budget. The system begins with a small governed boot, classifies the lifespan of the request, and loads only enough machinery to complete the current step. This keeps the writing system selective without weakening its controls.

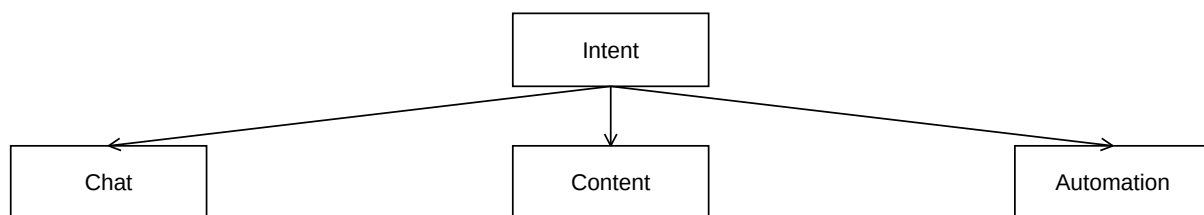
The public Writing Kit is intentionally bounded. It supports conversation, artifact creation, and tasks that have a defined completion condition. Long-running autonomous systems are outside this release boundary.

2. Three Operating Modes

Virtue7 exposes three modes to the user:

Chat | Create Content | Automate a Task

Chat means: think with me. It is conversational and need not create an artifact. Content means: make this. It produces a bounded artifact whose form, evidence, revision state, and approval status can be inspected. Automation means: perform this bounded job. It may use several steps or recur on a schedule, but each run has a completion condition and a terminal state.



Mode classifies lifespan before capability.

The distinction matters because recurrence is not persistence. A Friday report can run every Friday while each Friday run still begins, performs its defined work, records the result, and ends.

task = bounded work with a terminal state

3. Mode Is Not Capability

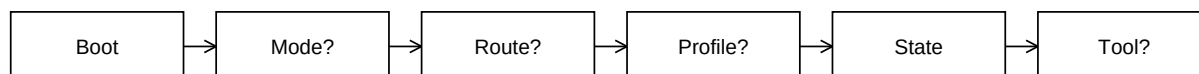
The three modes sit above the existing subject-matter capabilities. Writing, research, content creation, career materials, job search, legal preparation, continuation, review, and personalization can be invoked according to what the user is trying to accomplish.

This separation prevents the boot menu from becoming a catalogue of every feature. A user can choose a simple mode or simply state a substantive request. The runtime resolves the relevant capability and route underneath.

Mode answers how long the work lives. Capability answers what kind of work is being done. Profile answers what shape the artifact must take. Permission and tool contracts answer what external actions may occur.

4. Minimum-Sufficient Runtime

The runtime follows a simple loading principle: the smallest sufficient packet wins. The boot contains only `00_START_HERE.md`, `RUNTIME_MANIFEST.yaml`, and `MASTER_PROMPT.md`. A substantive task can then load one compiled route, one output profile if an artifact is being produced, compact relevant state, and a tool contract only when external action is actually required.



? = load only when the current step requires it

$B + M? + R? + P? + S_{min} + T_{min}$

Runtime components are loaded only when the active step needs them.

This architecture turns context management into a form of compilation. Capability files, quality gates, and route instructions are assembled into compact route packets ahead of time. Output profiles are similarly compiled with their effective form controls and final review gate. The runtime therefore avoids repeatedly loading adjacent routes, examples, histories, documentation, tests, and unrelated playbooks.

A handoff between capabilities should carry decisions and evidence rather than entire prior conversations. Completed work collapses into compact state before the next route is loaded.

5. Governed Language and Content

Virtue7's writing layer is governed by meaning, evidence, form, cadence, and source boundaries. A useful abstraction for canonical content is:

$$C = (K, U, E, P, Q)$$

Here K represents the core knowledge or meaning, U the units or sections, E evidence, P provenance, and Q unresolved questions or qualifications. Transformations should preserve the parts of that state that remain material to the new artifact.

Personal work can likewise be separated into authorized identity and voice evidence, role or organizational context, output preferences, and active project scope. Personalization is therefore scoped state rather than a license to infer private facts or imitate unrelated people.

Form is controlled by output profiles and Form Lock. Narrative Lock blocks unrequested bullets, numbered-list drift, fragment stacks, excessive headings, and pseudo-lists. A pseudo-list is a sequence of short standalone paragraphs or fragments that function as list items without list formatting. In continuous prose, those ideas must be developed and connected. When list structure is appropriate, they should be rendered as actual bullets or numbered items.

Evidence status is equally durable. A supplied fact, verified fact, credible report, quotation, interpretation, inference, reconstruction, allegation, disputed fact, theory, contradiction, and unknown are not interchangeable.

Expansion cannot create evidence. Compression cannot erase a qualification that changes the claim. Repurposing cannot silently upgrade certainty.

6. Bounded Automation

An automation is represented by a task contract. At minimum it defines an objective, inputs, allowed actions, tool and data scopes, success conditions, a completion condition, human gates, failure conditions, and required outputs or audit records.

The completion condition is essential because it gives the runtime a reason to stop. Consider the instruction: **Every Friday, review the approved support tickets and produce a weekly issue report.** The schedule may continue, but each execution is a fresh bounded run. The run reads the approved inputs, performs the review, creates the report, records any required receipts, and terminates.

This makes recurring work auditable. Each run can have its own inputs, outputs, tool receipts, state, and terminal status. A failed run does not silently become part of the next run, and a changed task contract can be versioned from a known boundary.

Automations may contain several capability steps. A research step can collapse its result into evidence and decisions before a writing step is loaded. The system does not need to retain every source and every intermediate instruction simultaneously.

7. Authority, Evidence, and Tool Truth

A capable host is not the same thing as an authorized action. Virtue7 therefore separates capability, permission, and approval. The environment may technically support an action while the current task does not permit it. A permitted action may still require a human approval event before a consequential transition occurs.

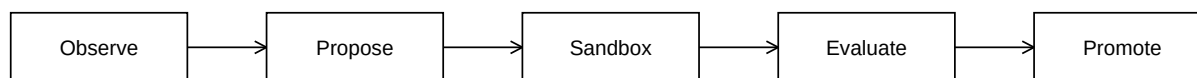
Tool-dependent work produces receipts tied to an exact target. The existence of a tool is not evidence that it was called. A successful tool call is not evidence that the resulting artifact was approved. The runtime must not claim publication, filing, messaging, export, retrieval, or other external action unless the host actually performed it and the required authorization was present.

Artifact approval is similarly exact. Human approval binds to an artifact identifier, version, and SHA-256. A material revision changes the artifact and therefore requires approval again. Export does not imply approval, and approval does not prove factual correctness.

8. Controlled Improvement

A governed system should be able to improve without silently rewriting the rules that protect the user. Virtue7 separates a protected governance layer from a mutable operating layer. Human authority, privacy boundaries, evidence rules, approval semantics, and tool truth belong to the protected layer. Prompt wording, workflow strategies, retrieval preferences, templates, and decomposition heuristics may be candidates for change.

observe -> propose -> sandbox -> evaluate -> promote or discard



Improvement is evaluated before promotion.

The important distinction is between proposing a better method and unilaterally changing production behavior. Candidate changes should be tested against profile fixtures, governance cases, adversarial cases, runtime contracts, and reproducible release checks before they become part of a published version.

9. Extension to More Powerful Execution

Virtue7 should remain useful when a host can do little more than read supplied files and produce text. It should also remain stable when connected to stronger execution systems. External capabilities therefore attach through explicit tool or execution contracts rather than becoming boot dependencies.

The Writing Kit owns intent, language quality, meaning preservation, evidence, state, permissions, approval, and mode selection. An execution layer may expand what the host can build or operate. The boundary states the action, inputs, target scope, authorization, expected receipt, failure behavior, and human gate. More powerful tools increase the action space; they do not weaken the authority model.

10. Conclusion

A useful AI writing system should not load every available mechanism merely because the model can use it. It should determine what the user is trying to do, choose the appropriate lifespan, and load only the machinery required for that step.

Virtue7 therefore begins with three choices: Chat, Create Content, and Automate a Task. The language system underneath them remains the same. Meaning, evidence, cadence, form, privacy, tool truth, project state, and human approval continue to govern the work. Minimum-sufficient loading keeps unrelated machinery dormant, while bounded automation provides repeatable execution without turning each scheduled job into an indefinite process.

The result is a compact public Writing Kit that can grow in usefulness without growing indiscriminately in active context. It becomes more capable by loading the right machinery at the right time, while keeping the human in control of evidence, authority, and release.